ONLINE MOVIE TICKET

Booking Platform

Solution Design & Architecture

XYZ Platform — B2B & B2C | Microservices | Cloud-Native | Java

Presented by: Sunil | Publicis Sapient Interview

CONFIDENTIAL — FOR INTERVIEW USE ONLY

Agenda

01 **Problem Statement & Goals**
B2B/B2C platform overview

02 **System Architecture**
Microservices, API Gateway, Cloud

03 **Functional Features & APIs**
REST API design, key scenarios

04 **Data Model & Database Design**
Schema, sharding, NoSQL

05 **Non-Functional Requirements**
Scalability, security, availability

06 **Platform Provisioning & DevOps**
Cloud, CI/CD, monitoring

07 **Technology Choices**
Key drivers & decisions

08 **Project Plan & Estimates**
Timeline & team structure

Problem Statement & Goals

PROBLEM

- Client needs an online movie ticket booking platform
- Must serve both B2B (theatre partners) and B2C (end customers)
- Theatre partners need digital tools to onboard & manage shows
- Customers need seamless browsing across cities, languages & genres
- Advance booking with an excellent UX is a core requirement

KEY GOALS

- Theatre onboarding with digital show management
- Multi-city, multi-language, multi-genre movie discovery
- Seat selection & advance booking
- Promotional offers (discounts, bulk booking)
- 99.99% platform availability & elastic scalability
- Secure payment gateway integration

AI Integration Recommendations

Smart Search & Recommendations

- OpenAI / AWS Bedrock for natural language movie search
- Collaborative filtering: 'Users like you also watched...'
- Content-based filtering by genre, language, cast preferences
- Personalized homepage feed per user segment

Dynamic Pricing Engine

- ML model predicts demand surge (big releases, weekends)
- Adjusts seat prices dynamically (like airline pricing)
- Amazon SageMaker or Azure ML for model training
- A/B test pricing strategies across user cohorts

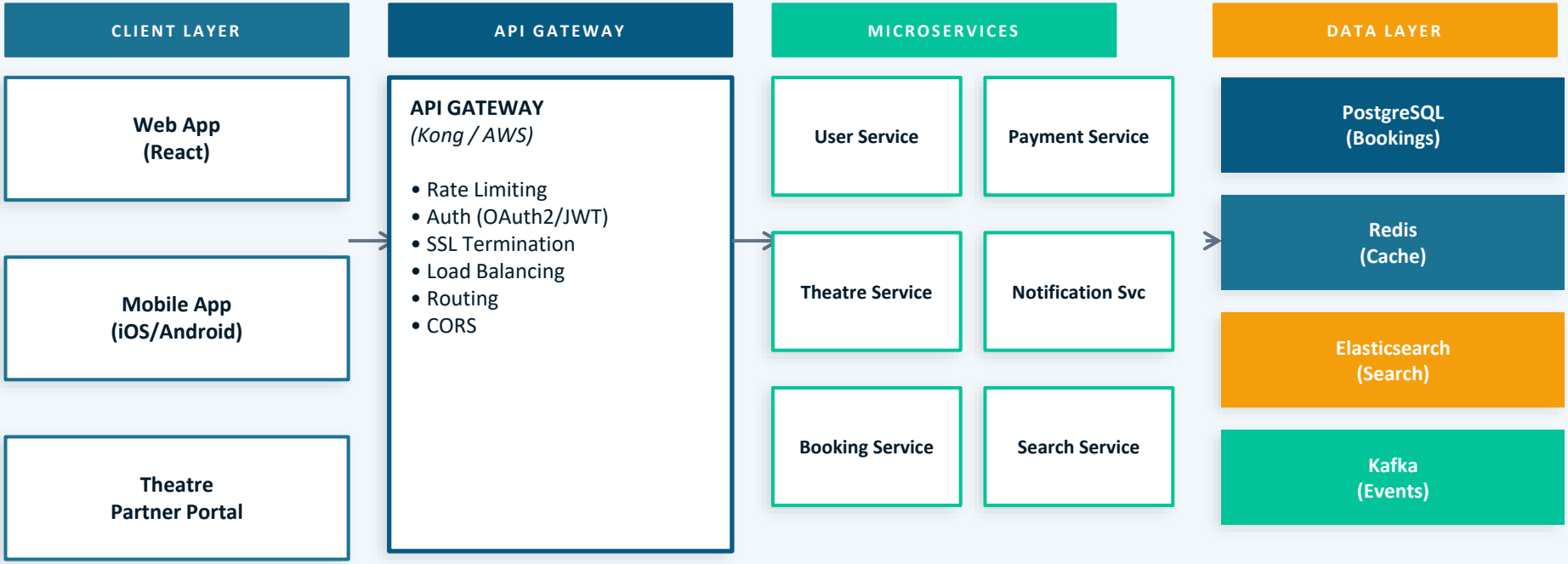
Fraud Detection

- ML anomaly detection on booking patterns
- Flag bulk bot bookings during high-demand releases
- Real-time scoring via SageMaker endpoint
- Reduces scalpers & black-market reselling

Chatbot / Virtual Assistant

- AWS Lex or Dialogflow for customer support bot
- Handle: show timings, booking status, cancellations
- Reduces support ticket volume by ~40%
- Multilingual support for regional theatres

System Architecture — Microservices



Data Model & Database Design

USER

```
user_id (PK)
name, email
phone, city
created_at
```

THEATRE

```
theatre_id (PK)
name, city
partner_id (FK)
total_screens
```

SHOW

```
show_id (PK)
movie_id (FK)
theatre_id (FK)
date, start_time
available_seats
```

BOOKING

```
booking_id (PK)
user_id (FK)
show_id (FK)
seat_ids[]
status, amount
created_at
```

MOVIE

```
movie_id (PK)
title, language
genre, duration
release_date
```

PAYMENT

```
payment_id (PK)
booking_id (FK)
gateway_ref
amount, status
paid_at
```

OFFER

```
offer_id (PK)
type (3rd_ticket,
afternoon)
discount_pct
city, theatre_id
```

SEAT

```
seat_id (PK)
show_id (FK)
row, number
category (A/B/C)
is_locked
```

API Contract & REST Design

KEY API ENDPOINTS

GET	`/api/v1/movies?city={city}&date={date}`
GET	`/api/v1/movies/{id}/shows`
GET	`/api/v1/theatres/{id}/seats/{showId}`
POST	`/api/v1/bookings`
PUT	`/api/v1/bookings/{id}/cancel`
POST	`/api/v1/theatres/{id}/shows`
DELETE	`/api/v1/theatres/{id}/shows/{showId}`
GET	`/api/v1/offers?city={city}&theatreId={id}`

BEST PRACTICES

Versioning	URI versioning /api/v1/
Pagination	?page=0&size=20&sort=date,asc
Error Codes	400, 401, 403, 404, 409, 429, 500
HATEOAS	Links in responses for navigation
Security	OAuth2 + JWT Bearer tokens
Swagger	OpenAPI 3.0 documentation
Idempotency	PUT/DELETE idempotent by design

Code Implementation — Read Scenarios

Scenario 1: Browse Theatres Running a Show

```
// GET /api/v1/movies/{movieId}/shows?city=Mumbai&date=2024-01-15
@GetMapping("/movies/{movieId}/shows")
public ResponseEntity<List<ShowDTO>> getShows(
    @PathVariable Long movieId,
    @RequestParam String city,
    @RequestParam @DateTimeFormat(iso=DATE) LocalDate date) {
  List<Show> shows = showService.findByMovieCityDate(movieId,
city, date);
  return ResponseEntity.ok(showMapper.toDTOList(shows));
}
```

Scenario 2: Fetch Applicable Offers

```
// Offer Rules Engine
public OfferResult applyOffers(BookingRequest req) {
  BigDecimal discount = BigDecimal.ZERO;
  // Rule 1: 50% on 3rd ticket
  if (req.getSeatCount() >= 3)
    discount = discount.add(req.getUnitPrice()
        .multiply(new BigDecimal("0.5")));
  // Rule 2: 20% for afternoon show
  if (req.getShowTime().isBefore(LocalTime.of(17,0)))
    discount = discount.add(req.getTotalPrice()
        .multiply(new BigDecimal("0.2")));
  return new OfferResult(discount, buildDescription());
}
```

DESIGN PATTERNS APPLIED IN CODE

Strategy Pattern:	OfferStrategy interface with AfternoonDiscountStrategy, ThirdTicketStrategy implementations — swap rules without changing BookingService	**Builder Pattern:**	BookingRequest.builder().movieId().showId().seats().build() — clean object construction
Repository Pattern:	ShowRepository extends JpaRepository — abstracts DB queries from business logic	**DTO Pattern:**	ShowDTO separates API contract from JPA entity — API changes don't break DB schema

Code Implementation — Write Scenarios

Theatre: Create / Update / Delete Show

- POST /api/v1/theatres/{id}/shows → Creates new show, validates no overlap in screen timing
- PUT /api/v1/shows/{showId} → Updates show details, invalidates Redis cache for that show
- DELETE /api/v1/shows/{showId} → Soft delete (status=CANCELLED), notifies booked users via Kafka event
- Role guard: only THEATRE_ADMIN JWT role can call these endpoints (@PreAuthorize)

Theatre: Allocate & Update Seat Inventory

- POST /api/v1/shows/{showId}/seats/allocate → Bulk insert seat rows (A1–A20, B1–B20)
- PUT /api/v1/shows/{showId}/seats/{seatId} → Update category (GOLD/SILVER) or price
- Uses @Transactional with batch insert (JdbcTemplate batch for performance)
- Seat layout stored as JSON in PostgreSQL for flexible screen configurations

Bulk Booking

- POST /api/v1/bookings/bulk → Accept array of BookingRequest objects
- Each sub-booking processed in parallel using CompletableFuture
- All-or-nothing: if any seat fails, entire bulk booking is rolled back (SAGA compensation)
- Use case: corporate bookings, school trips, event group purchases
- Rate limited at gateway: max 20 seats per bulk request to prevent abuse

Bulk Cancellation

- PUT /api/v1/bookings/bulk-cancel → Accept array of bookingIds
- Validate: cancellation allowed only up to 2 hours before show
- Refund initiated per booking via Payment Service (async via Kafka)
- Seats released back to inventory immediately (synchronous)
- Audit log: every cancellation stored with timestamp and reason

Scenario: Book Movie Tickets (SAGA Pattern)

1 Create Booking

POST /api/v1/bookings
Status: PENDING

→

2 Reserve Seats

Lock seats in DB
(optimistic locking)

→

3 Apply Offers

50% on 3rd ticket
20% on afternoon

→

4 Process Payment

Payment gateway
(Stripe/Razorpay)

→

5 Confirm Booking

Status: CONFIRMED
Send notifications

SAGA COMPENSATION (Rollback) — If Payment Fails

- Payment Service → publishes PaymentFailed event to Kafka
- Booking Service → listens, updates status to FAILED
- Seat Service → releases locked seats back to inventory
- Notification Service → sends failure notification to user

Theatre Integration & Localization

THEATRE INTEGRATION PATTERNS

New Digital Theatres

REST API onboarding portal. Self-service registration. Webhook subscription for real-time show updates.

Legacy IT Systems

Adapter / Anti-Corruption Layer translates their proprietary format. Scheduled batch pull via SFTP or SOAP-to-REST bridge. Apache Camel for integration routing.

Real-time Sync

Theatre pushes show/seat changes via webhook → our Integration Service validates & publishes to Kafka → Show Service consumes & updates DB + Elasticsearch.

Error Handling

Dead Letter Queue for failed messages. Retry with exponential backoff. Alert on 3 consecutive failures.

LOCALIZATION (i18n / L10n)

Language Support

Spring MessageSource for UI strings. Movie metadata stored in multiple languages (title_en, title_hi, title_ta). Accept-Language header drives response language.

Currency & Pricing

Multi-currency support (INR, USD, AED). Currency stored separately from amount. Exchange rates via scheduled sync from open exchange rates API.

Date/Time Zones

All timestamps stored in UTC in DB. Converted to local timezone on display. Show times displayed in theatre's local timezone.

Regional Content

Movie catalogue filtered by region/language preference. Tamil movies shown to TN users, Marathi to Maharashtra. Elasticsearch geo-filter by city/state.

Non-Functional Requirements

Scalability

Horizontal scaling via Kubernetes HPA. Stateless services behind load balancer. DB sharding by city/region. CDN for static assets.

Availability 99.99%

Multi-AZ deployment on AWS. Circuit breaker (Resilience4j). Active-active DB replication. Health checks & auto-recovery.

Caching

Redis for seat availability, show listings. TTL-based expiry. Cache-aside pattern. Distributed cache with write-through for bookings.

Security (OWASP)

OAuth2 + JWT. Input validation. SQL injection prevention. Rate limiting at gateway. HTTPS/TLS everywhere. XSS, CSRF protection.

Transactions

SAGA pattern for distributed transactions. Optimistic locking for seat reservation. Idempotency keys for payment retries.

Multi-tenancy

Theatre partner isolation via tenant_id. Row-level security in PostgreSQL. Separate namespaces in Kubernetes per partner tier.

Platform Monetization Strategy

Convenience Fee per Ticket

Charge ₹20–₹50 per ticket as platform fee. Core revenue stream. Configurable per city/theatre tier. Shown transparently at checkout.

~₹30 avg / ticket

Theatre Subscription Plans

Basic (free, limited listings), Standard (₹2,999/mo, analytics dashboard), Premium (₹7,999/mo, priority placement, AI insights).

SaaS B2B revenue

Advertisement & Promotions

Banner ads on movie listing pages. Promoted show placement (theatres pay for top-of-list). Sponsored genres or 'Editor's Pick' labels.

₹X per impression

Dynamic Pricing Commission

Revenue share on AI-driven dynamic pricing uplift. Platform earns 10% of price premium above base ticket price during surge periods.

10% of uplift

Data & Analytics Products

Sell anonymized audience insights to movie studios and distributors. Which demographics buy which genres, regional preferences.

B2B data product

Loyalty & Gift Cards

Float income on unredeemed gift cards. Corporate bulk gift card sales. Loyalty points partnerships with banks and credit cards.

Float + B2B deals

Compliance & Regulatory Requirements

PCI-DSS (Payment Card Industry)

- Never store raw card data — use tokenization via Stripe/Razorpay
- All payment flows over TLS 1.3 minimum
- Regular vulnerability scans and penetration testing
- Access to cardholder data restricted by role (least privilege)
- Quarterly PCI compliance audits

GDPR / PDPA (Data Privacy)

- Explicit consent collected at registration for data usage
- Right to erasure: DELETE /api/v1/users/{id}/data wipes PII
- Data minimization: only collect what's needed
- Anonymize analytics data — no PII in reports or Kafka events
- Data residency: Indian user data stays in AWS ap-south-1

IT Act / CERT-In (India)

- Report cybersecurity incidents to CERT-In within 6 hours
- Maintain logs for 180 days (CloudWatch log retention policy)
- KYC for theatre partners via Aadhaar/GST verification API
- Comply with intermediary liability guidelines

Accessibility (WCAG 2.1)

- Screen reader support for visually impaired users
- Keyboard-navigable seat selection UI
- High contrast mode and font size options
- Mandatory for government/public sector theatre contracts

Technology Choices & Decisions

Layer	Technology	Why / Key Driver
Backend	Java 17 + Spring Boot 3	Mature ecosystem, reactive support, team expertise
API Gateway	Kong / AWS API Gateway	Rate limiting, auth plugins, observability out of box
Database	PostgreSQL (Aurora)	ACID compliance, partitioning, JSON support
Cache	Redis Cluster	Sub-ms latency, pub/sub, seat locking with TTL
Messaging	Apache Kafka (MSK)	High-throughput, event sourcing, SAGA choreography
Search	Elasticsearch	Full-text search, geo queries, faceted filtering
Containers	Docker + Kubernetes (EKS)	Auto-scaling, blue/green deployments, self-healing
Monitoring	ELK + Grafana + X-Ray	Distributed tracing, log aggregation, dashboards
Auth	Keycloak / AWS Cognito	OAuth2/OIDC, JWT, multi-tenant identity

COTS Enterprise Systems

Commercial Off-The-Shelf systems we integrate rather than build from scratch:

Keycloak / AWS Cognito

Identity & Access Mgmt

Enterprise-grade OAuth2/OIDC. SSO across B2B portal and B2C app. MFA, social login, LDAP federation for large theatre chains.

Stripe / Razorpay

Payment Processing

PCI-DSS compliant out of the box. Handles card tokenization, refunds, recurring billing for subscriptions. Saves 6–12 months of payment engineering.

Twilio / AWS SNS

Notification Gateway

SMS OTPs, booking confirmations across 200+ countries. Email via SendGrid/SES. Push via Firebase Cloud Messaging. No need to manage carrier integrations.

Salesforce / HubSpot

CRM for Theatre Partners

Manage B2B theatre partner relationships, contracts, support tickets. Sales pipeline for onboarding new theatre chains.

Apache Camel / MuleSoft

Integration / ESB

Connect to legacy theatre IT systems using SOAP, SFTP, FTP adapters. Pre-built connectors reduce integration time from weeks to days.

Datadog / New Relic

APM & Observability

Enterprise-grade application performance monitoring. Distributed tracing, anomaly detection, SLA reporting dashboards for ops team.

Hosting Solution & Infrastructure Sizing

CLOUD HOSTING STRATEGY

Primary: AWS (ap-south-1)

Mumbai region for low latency in India. Primary traffic and data residency.

DR: AWS (ap-southeast-1)

Singapore as disaster recovery region. RDS Aurora Global DB for sub-second RPO. Route53 failover DNS.

Multi-Cloud (Phase 2)

GCP for ML workloads (BigQuery, Vertex AI). Avoids vendor lock-in. Anthos for hybrid K8s management.

CDN: CloudFront

Static assets, movie posters, trailers cached at edge. Sub-50ms response globally.

On-Premise Option

For large theatre chains with data sovereignty requirements. Deploy containerized services on their infra with our Helm charts.

INITIAL SIZING (Launch)

Component	Spec	Count
API Gateway	EKS t3.medium	3 pods
User Service	EKS t3.medium	2 pods
Booking Service	EKS t3.large	3 pods
Payment Service	EKS t3.large	2 pods
Search Service	EKS t3.medium	2 pods
PostgreSQL Aurora	db.r6g.large	1W + 1R
Redis ElastiCache	cache.r6g.large	3 nodes
Kafka MSK	kafka.m5.large	3 brokers
Elasticsearch	r6g.large.search	3 nodes

Platform Provisioning & DevOps

CLOUD ARCHITECTURE (AWS)

- EKS (Kubernetes) — Service orchestration
- API Gateway + ALB — Traffic management
- RDS Aurora (Multi-AZ) — Transactional DB
- ElastiCache Redis — Caching layer
- MSK (Kafka) — Event streaming
- Elasticsearch Service — Search
- S3 + CloudFront — Static assets & CDN
- Route53 + WAF — DNS & security
- CloudWatch + X-Ray — Monitoring

CI/CD PIPELINE

Code	GitHub branching strategy (GitFlow)
Build	Maven build → Docker image
Test	JUnit, Mockito, Integration tests
Scan	SonarQube, OWASP dependency check
Deploy	Helm charts → EKS (Blue/Green)
Monitor	ELK Stack, Grafana, Alerting

Release Management, Geos & KPIs

RELEASE MANAGEMENT ACROSS GEOS

- Feature flags (LaunchDarkly) — enable features per region without code deployment
- Phased rollout: Tier-1 cities (Mumbai, Delhi, Bangalore) first → Tier-2 → International
- Blue/Green deployment per region — zero downtime. Canary: 5% → 25% → 100% traffic shift
- DB migrations via Flyway — version-controlled, run automatically in pipeline
- Rollback strategy: Git tag revert → Helm rollback → DB migration down script

PLATFORM KPIs

Availability:	99.99% uptime SLA
API Latency (p95):	< 300ms
Booking Success Rate:	> 98%
Seat Lock Conflicts:	< 0.5%
Payment Success Rate:	> 99.2%
Time to Onboard Theatre:	< 2 business days

INTERNATIONALIZATION (i18n) & MULTI-GEO

- Each geo has its own AWS region deployment (India → ap-south-1, Middle East → me-south-1, SEA → ap-southeast-1)
- Spring Boot i18n: messages.properties per locale (messages_hi.properties, messages_ta.properties). Locale resolved from Accept-Language header
- Right-to-left language support (Arabic, Urdu) via CSS dir=rtl, handled at frontend layer
- Tax rules per country: GST 18% in India, VAT in UAE — pluggable TaxStrategy per region

Project Plan & Estimates

Phase 1 Foundation (6 wks)	Phase 2 Core Features (8 wks)	Phase 3 Payments & NFR (6 wks)	Phase 4 Launch Readiness (4 wks)
• Infra setup, CI/CD pipeline • User & Auth services • Theatre onboarding API • Core DB schema	• Movie & Show management • Search & Browse APIs • Booking + Seat selection • SAGA transactions	• Payment gateway integration • Offer/discount engine • Caching & performance • OWASP security hardening	• Load testing & tuning • Monitoring & alerts • Multi-city rollout • UAT & Go-live

Team: 2 Lead Devs | 4 Backend Devs | 1 DevOps | 1 QA | 1 Architect — Total: ~24 weeks end-to-end

Stakeholder Management

STAKEHOLDER MAP

Theatre Partners

B2B Client

Ease of onboarding, revenue visibility

End Customers

B2C User

Seamless booking, fair pricing

Product Owner

Internal

Feature delivery, backlog priority

Engineering Lead

Internal

Tech debt, scalability, team velocity

DECISIONS & ACTIONS TAKEN FOR CLOSURE

Decision: Microservices vs Monolith

Context: Engineering preferred monolith for speed; Product wanted scalability from day 1. → Resolution: Modular monolith for Phase 1, microservices decomposition in Phase 2. Documented as ADR (Architecture Decision Record). Signed off by CTO.

Decision: Theatre API integration standard

Context: Theatre partners requested different integration methods (REST, SFTP, webhooks). → Resolution: Defined standard integration contract. Built adapter layer for legacy. Timeboxed onboarding support to 5 business days per partner. Partner SLA document signed.

Decision: Dynamic pricing rollout

Context: Some theatre partners objected to dynamic pricing — feared customer backlash. → Resolution: Made opt-in for theatres. Pilot with 2 willing partners first. Shared data showing 18% revenue uplift in pilot. Gradually adopted by 70% of partners.

Technology & Delivery Management

OVERALL TECHNOLOGY MANAGEMENT

- Architecture Decision Records (ADRs) for every major tech choice
- Tech radar: Adopt / Trial / Assess / Hold categories updated quarterly
- SonarQube quality gates enforced in CI/CD — no merge below 80% coverage
- Dependency upgrades tracked via Dependabot with monthly upgrade sprints
- Tech debt backlog maintained separately, 20% of each sprint allocated to it

ENABLING TEAM & INTRODUCING EFFICIENCIES

- Inner source: shared libraries for JWT validation, error handling, logging
- Golden path templates: Spring Boot microservice scaffold via Maven archetype
- Weekly tech talks: knowledge sharing across teams (30 min, recorded)
- Pair programming for complex SAGA and payment integration modules
- GitHub Copilot for boilerplate — estimated 25% reduction in coding time

DELIVERY PLANNING & ESTIMATES

Methodology:	SAFe Agile — 2-week sprints, PI Planning every 10 sprints (quarterly roadmap alignment)
Estimation:	Story points (Fibonacci). Planning poker for complex stories. Historical velocity used for sprint capacity.
Risk Management:	Top risks logged in RAID log (Risks, Assumptions, Issues, Dependencies). Reviewed every sprint.
Definition of Done:	Code reviewed + merged + unit tested + SonarQube green + deployed to staging + smoke tested.

Thank You

Ready to discuss the solution in detail

- Microservices architecture with SAGA, CQRS, and Event-Driven patterns
- Cloud-native on AWS with 99.99% availability design
- Secure, scalable, multi-tenant platform for B2B and B2C
- Comprehensive DevOps pipeline with Blue/Green deployments

CONFIDENTIAL — FOR INTERVIEW USE ONLY